LAB 14 Rendering performance, animations and Android native channels

Unit 5 • 3-hour guided laboratory • CLO-5

Learning goal. Use a controlled animation without resource leaks, construct performant scrollable widgets and obtain real Android hardware information through a MethodChannel.

Core concepts. AnimatedContainer, AnimationController, SingleTickerProviderStateMixin, ListView.builder, debug profiling, Kotlin/Flutter platform channels.

Before you start. Keep a physical Android device attached. Task 2 requires a specifically named Kotlin-package Android project; see its command.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab14
cd mad_lab14
```

Solved task 1 | AnimationController with animated card and efficient list

Problem and objective. Make a button transition the size and color of a banner while an explicit AnimationController animates a progress line. Render a lazy 1000-row list.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: AnimatedPage()));

class AnimatedPage extends StatefulWidget {
  const AnimatedPage({super.key});
  @override
  State<AnimatedPage> createState() => _AnimatedPageState();
}

class _AnimatedPageState extends State<AnimatedPage>
    with SingleTickerProviderStateMixin {
  late final AnimationController controller;
  bool expanded = false;

  @override
  void initState() {
    super.initState();
    controller = AnimationController(
      vsync: this, duration: const Duration(milliseconds: 500));
  }

  @override
  void dispose() { controller.dispose(); super.dispose(); }

  void toggle() {
    setState(() => expanded = !expanded);
    if (expanded) { controller.forward(); }
    else { controller.reverse(); }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Flutter animation lab')),
    body: Column(children: [
      AnimatedContainer(
        duration: const Duration(milliseconds: 500),
        margin: const EdgeInsets.all(12),
        height: expanded ? 140 : 75,
        width: double.infinity,
        decoration: BoxDecoration(
          color: expanded ? Colors.teal : Colors.indigo.shade200,
          borderRadius: BorderRadius.circular(14)),
        child: const Center(child: Text('Tap below to animate')),
      ),
      AnimatedBuilder(animation: controller,
        builder: (_, __) => LinearProgressIndicator(
          value: controller.value)),
      FilledButton(onPressed: toggle,
        child: Text(expanded ? 'Collapse' : 'Expand')),
      Expanded(child: ListView.builder(
```

```
        itemCount: 1000,
        itemBuilder: (_, i) => ListTile(
          leading: const Icon(Icons.bolt),
          title: Text('Efficient row ${i + 1}')),
      )),
    ]),
  );
}
```

How the solution works. AnimatedContainer interpolates its properties implicitly when the state changes. AnimationController advances an explicit numeric progress value, and AnimatedBuilder rebuilds only its progress subtree. The ticker and controller are disposed to prevent leaked animation resources. ListView.builder lazily builds visible rows, making 1000 records feasible.

Expected output. The card expands and changes color; a progress bar animates in sync and the large list scrolls.

Check your work. Use Flutter DevTools Performance and test in profile mode on a real phone if you investigate frame timing; debug-mode timing is not representative.

Solved task 2 | Kotlin MethodChannel to read battery percentage

Problem and objective. Create a Flutter project with an exact Android package name so the native file path matches. A Dart MethodChannel requests the battery percentage from Kotlin's BatteryManager.

TERMINAL — NEW PROJECT (run outside mad_lab14)

```
flutter create --platforms=android --org edu.cui.sahiwal mad_native_demo
cd mad_native_demo
# Open this folder in Android Studio before editing MainActivity.kt
```

android/app/src/main/kotlin/edu/cui/sahiwal/mad_native_demo/MainActivity.kt

```
package edu.cui.sahiwal.mad_native_demo

import android.content.Intent
import android.content.IntentFilter
import android.os.BatteryManager
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.MethodChannel

class MainActivity : FlutterActivity() {
    private val channel = "edu.cui.sahiwal/device"

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger, channel)
            .setMethodCallHandler { call, result ->
                if (call.method == "batteryLevel") {
                    val battery = registerReceiver(
                        null, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
                    val level = battery?.getIntExtra(
                        BatteryManager.EXTRA_LEVEL, -1) ?: -1
                    val scale = battery?.getIntExtra(
                        BatteryManager.EXTRA_SCALE, -1) ?: -1
                    if (level >= 0 && scale > 0) {
                        result.success(level * 100 / scale)
                    } else {
                        result.error("UNAVAILABLE", "Battery not available", null)
                    }
                } else {
                    result.notImplemented()
                }
            }
    }
}
```

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

void main() => runApp(const MaterialApp(home: BatteryPage()));

class BatteryPage extends StatefulWidget {
  const BatteryPage({super.key});
  @override
  State<BatteryPage> createState() => _BatteryPageState();
}
```

```
class _BatteryPageState extends State<BatteryPage> {
  static const channel = MethodChannel('edu.cui.sahiwal/device');
  String battery = 'Not read yet';

  Future<void> readBattery() async {
    try {
      final value = await channel.invokeMethod<int>('batteryLevel');
      if (mounted) setState(() => battery =
          value == null ? 'Unknown' : '$value%');
    } on PlatformException catch (e) {
      if (mounted) setState(() => battery = 'Native error: ${e.code}');
    } on MissingPluginException {
      if (mounted) setState(() => battery = 'Native channel missing');
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Android battery channel')),
    body: Center(child: Column(mainAxisSize: MainAxisSize.min,
      children: [
        Text('Battery: $battery',
          style: Theme.of(context).textTheme.headlineSmall),
        FilledButton(onPressed: readBattery,
          child: const Text('Read Android battery')),
      ],
    )),
  );
}
```

TERMINAL — native code requires full rebuild

```
flutter clean
flutter pub get
flutter run
```

How the solution works. MethodChannel communicates between the Dart UI and the Android embedding through a string channel name and a method name. Kotlin reads the sticky ACTION_BATTERY_CHANGED broadcast and translates the level/scale pair into a percentage. A native source edit requires a complete rebuild; Dart hot reload does not replace compiled Kotlin.

Expected output. The phone reports a battery percentage close to Android settings; no dangerous runtime permission is necessary for this broadcast.

Check your work. Check that the package statement matches the directory and Gradle application namespace, and that both sides spell channel/method identically.

Scenario-based practice — unsolved

Scenario 1. Animated attendance tile. Design a reduced-motion-safe attention animation for late arrivals. Explain why repeating animations may be distracting and how to honor accessibility settings.

Scenario 2. Hardware feature request. A campus app must read accelerometer values and upload aggregated readings. Identify permission, privacy, platform-interface, battery and sampling concerns.

Technical viva — justify your reasoning

1. Why does AnimationController need a vsync provider?
2. How is a repaint different from a build and layout pass?
3. Why is profile mode better than debug mode for benchmarking?
4. What causes MissingPluginException?
5. Why can Dart hot reload not apply modified Kotlin bytecode?
6. What is the impact of a channel name mismatch?

Submission and self-check

Submit a smooth animation clip, a battery reading on a real Android device, Kotlin and Dart source, and a note on profiling versus debugging.

CSC303 / NOVEMBER 2026 REVISION

LAB 14 EXTENSION | Evidence-based performance and native bridge

Students should distinguish a visual animation demonstration from a performance measurement. Running at 60 Hz is not proof that a screen maintains its frame budget under stress.

Performance experiment

• Run the animation demo on a real phone in profile mode. Record device model, screen refresh rate, Flutter version, and which performance tool was used.

```
flutter devices
flutter run --profile -d ANDROID_DEVICE_ID
# Inspect frame timings in Flutter DevTools Performance.
```

• Capture a trace with the animation running, then another after reducing unnecessary rebuilds. Compare representative frame timings; do not invent FPS values.

• Turn on device reduced-motion preference and verify that distracting repeated motion is disabled or replaced by an accessible representation.

MethodChannel correctness checks

• Test the exact channel identifier and method identifier on both the Kotlin and Dart sides.

• Verify `level` and `scale` are present and a positive scale is used before computing a battery percentage.

• Change native Kotlin code; perform a full rebuild rather than relying on Dart hot reload.

• Demonstrate a graceful failure if a native plugin/channel is missing, without an uncaught crash.

Optional extension

Wrap the platform-channel call behind a small interface. Create a fake in a unit test that returns 42 and one that throws an error. This illustrates how architecture and tests from Lab 13 transfer to Android integration.

Submission

Provide one annotated profiling screenshot, one accessibility demonstration, and the Dart/Kotlin source with reproducible device and SDK details.